Use Case Diagram

Introduction:

What is UML?

UML , short for Unified Modeling Language, is a standardized modeling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems.

Types of UML Diagrams -

*Use case Diagram
*Class Diagram
*Activity Diagram
*Sequence Diagram
*State Diagram
*Deployment Diagram

What is USE CASE

USE CASE:

* A use case is a pattern of behavior, the system exhibits.

* The use cases are sequence of actions that the user takes on a system to get particular target.

* USE CASE is dialogue between an actor and the system.

Introduction

Use-cases are descriptions of the functionality of a system from a user perspective.

- Depict the behaviour of the system, as it appears to an outside user.
- Describe the functionality and users (actors) of the system.
- Show the relationships between the actors that use the system, the use cases (functionality) they use, and the relationship between different use cases.
- Document the scope of the system.
- Illustrate the developer's understanding of the user's requirements.

A use case diagram contains four main components here we gonna discuss about it and others components also;

1 #Use Case

A use case represents a user goal that can be achieved by accessing the system or software application.

2 #Actor

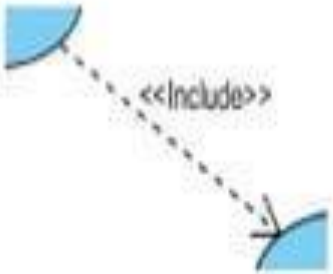
Actors are used to represent the users of system, actors can actually be anything that needs to exchange information with the system.

3 #System

The scope of a system can be represented by a system (shape), or sometimes known as a system boundary. The use cases of the system are placed inside the system shape, while the actor who interact with the system are put outside the system.

4. Connectors

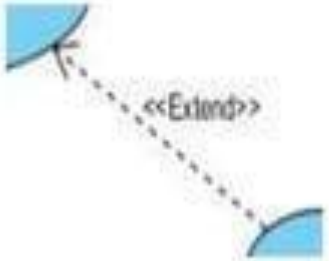
#Include



An include relationship specifies how the behavior for the inclusion use case is inserted into the behavior defined for the base use case.

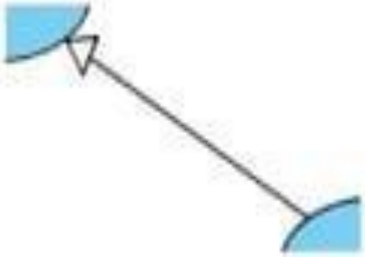
#Extend

1



An extend relationship specifies how the behavior of the extension use case can be inserted into the behavior defined for the base use case.

#Generalization



A generalization relationship is used to represent inheritance relationship between model elements of same type.

ACTOR

* An actor is some one or something that must interact with the system under development.

* Actors can be human or automated systems.

* Actors are not part of the system.

* UML notation for actor is stickman, shown below.

* Actors carry out use cases and a single actor may perform more than one use cases.

* Actors are determined by observing the direct uses of the system.

Primary and Secondary Actors

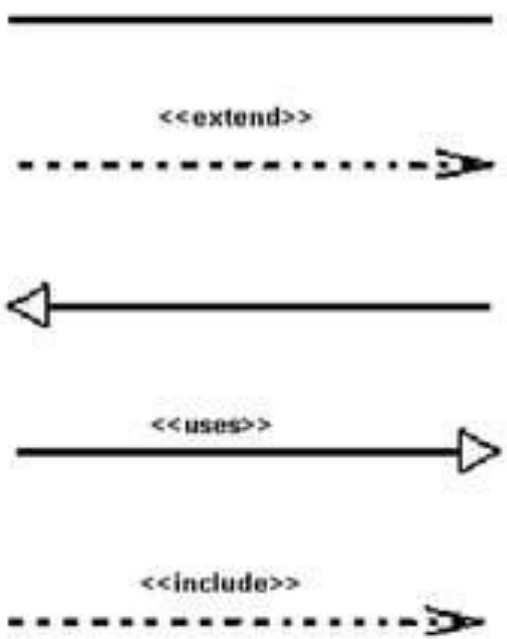
- **Primary Actor**
  - Acts on the system.
  - Initiates an interaction with the system.
  - Uses the system to fulfill his/her goal.
  - Events Something we don't have control over.

- **Secondary Actor**
  - Is acted on/invoked/used by the system.
  - Helps the system to fulfills its goal.
  - Something the system uses to get its job done.

Relationship

- Relationship is an association between use case and actor.
- There are several Use Case relationships:

  - Association
  - Extend
  - Generalization
  - Uses
  - Include



Linking Use Cases

- Association relationships.
- Generalization relationships.
- One element (child) "is based on" another element (parent).
- Include relationships.
- One use case (base) includes the functionality of another (inclusion case).
- Supports re-use of functionality.
- Extend relationships.
- One use case (extension) extends the behavior of another (base).

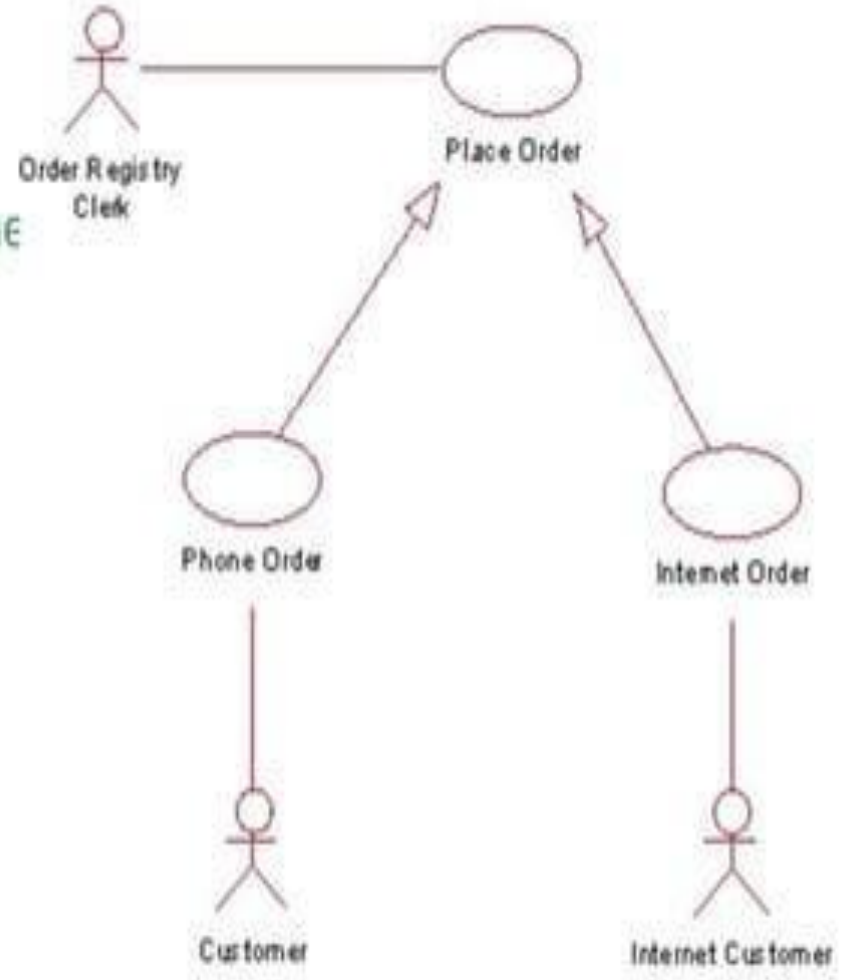
Generalization

- Generalization is a relationship between a general use case and a more specific use case that inherits and extends features to it.
- use cases that are specialized versions of other use cases.
- It is shown as a solid line with a hollow arrow point.

Generalization Example

* The actor Order Registry Clerk can instantiate the general use case Place Order.

* Place Order can also be specialized by the use cases Phone Order or Internet Order.



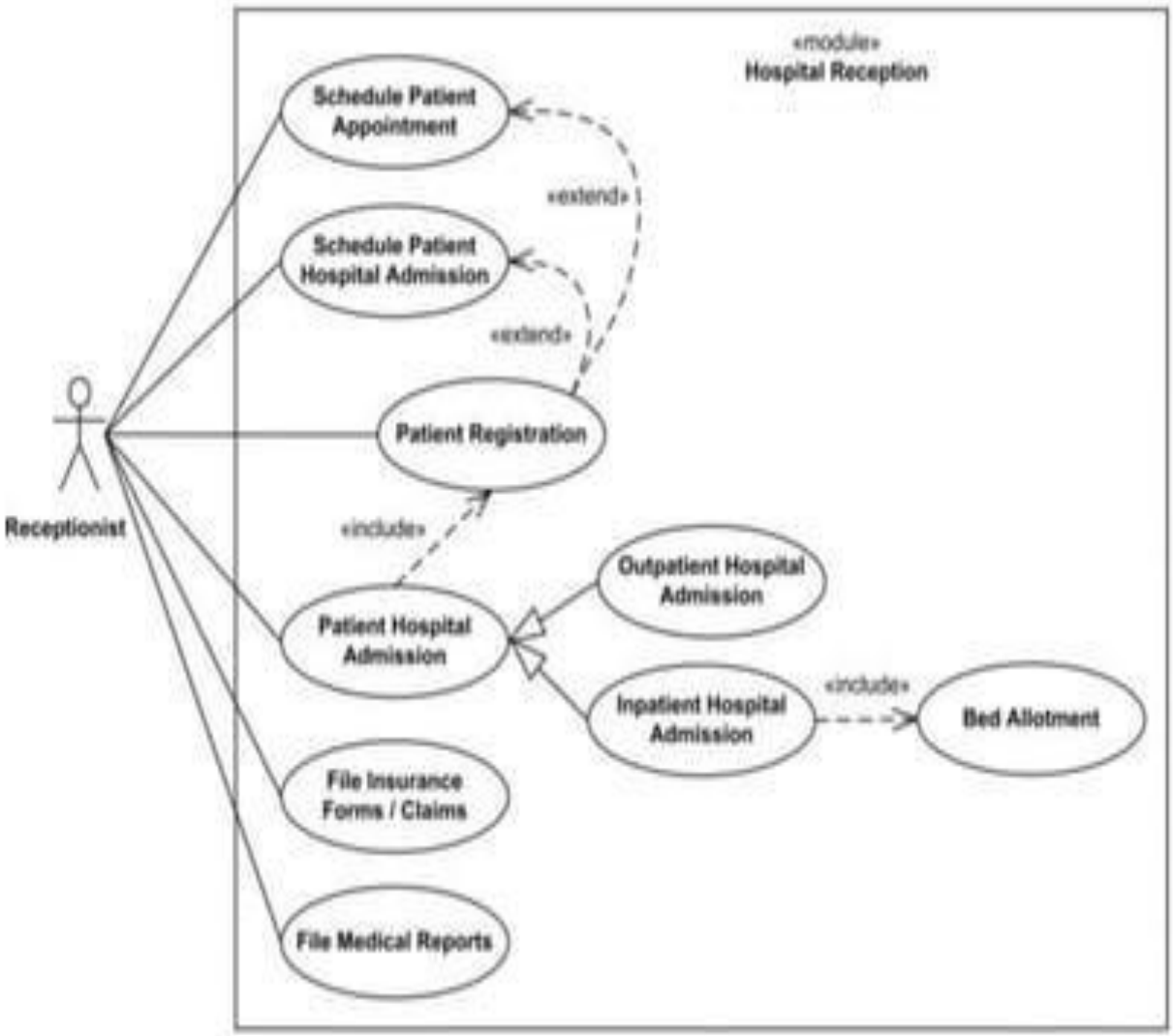
Include

- The base use case explicitly incorporates the behavior of another use case at a location specified in the base.

- The included use case never stands alone. It only occurs as a part of some larger base that includes it.

Base Use Case	Included Use Case	Relationship	Explanation
Withdraw Funds	Verify PIN	Withdraw Funds $\ll \text{include} \gg$ Verify PIN	To Withdraw Funds, the system must execute the Verify PIN process.
Check Balance	Verify PIN	Check Balance $\ll \text{include} \gg$ Verify PIN	To Check Balance, the system must execute the Verify PIN process.

Include Example



Extend

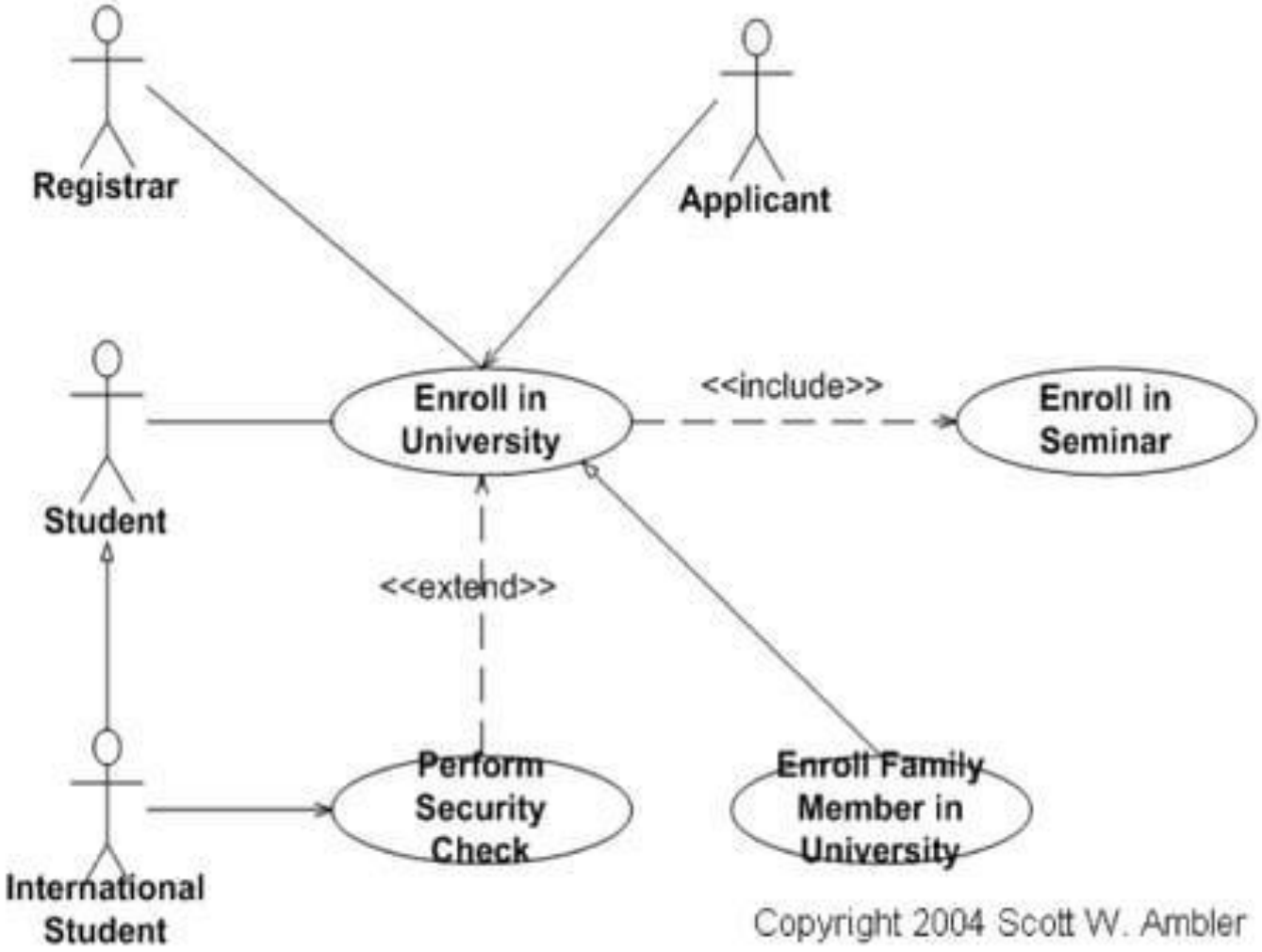
- The base use case implicitly incorporates the behavior of another use case at certain points called extension points.

- The base use case may stand alone, but under certain conditions its behavior may be extended by the behavior of another use case.

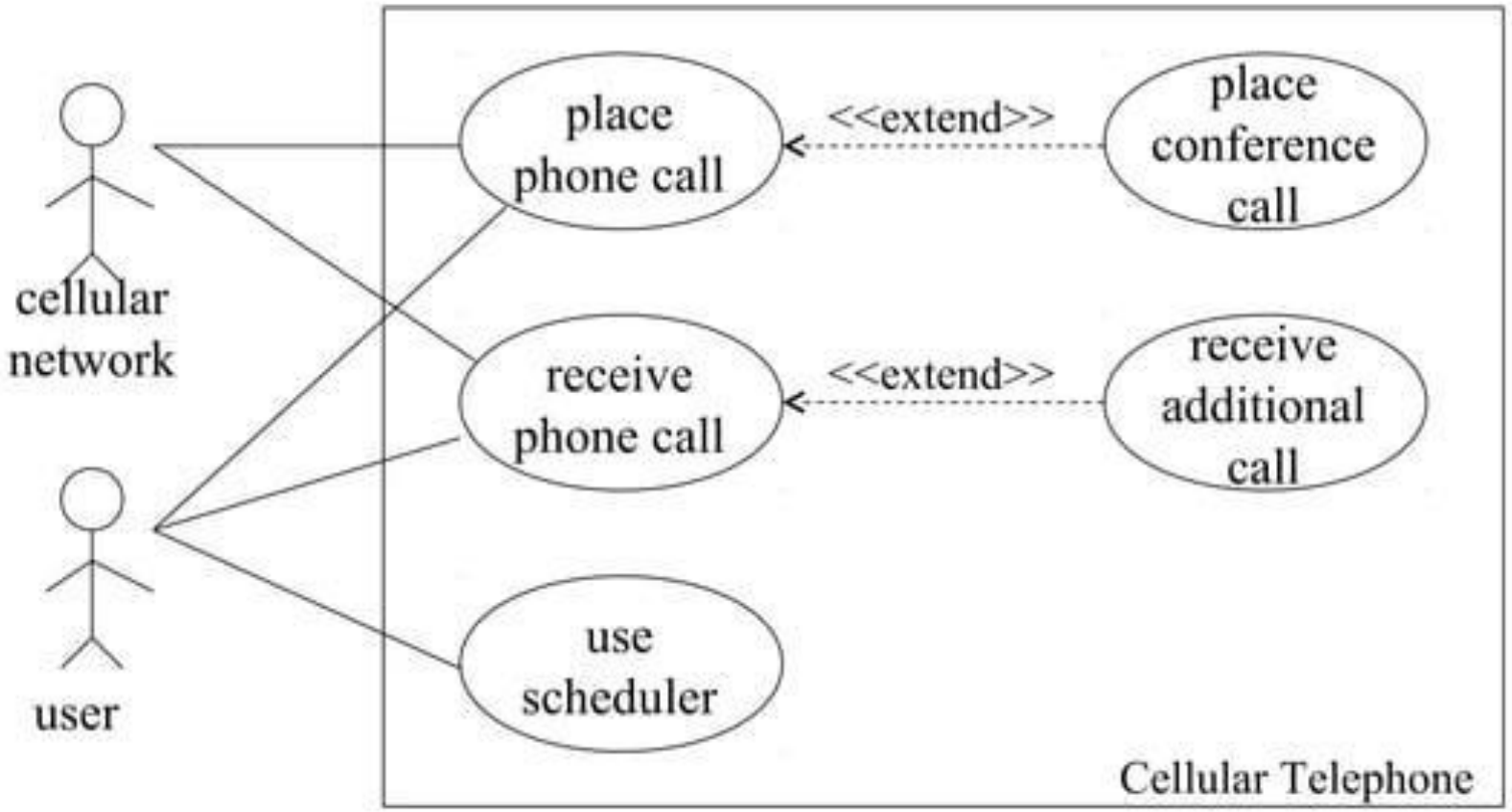
Extend

Base Use Case	Extending Use Case	Condition	Relationship
Withdraw Funds	Print Receipt	The user selects the "Yes" option for a receipt.	Withdraw Funds $\ll \text{extend} \gg$ Print Receipt
Process Transaction	Notify Fraud Department	The account is overdrawn or the amount is suspicious.	Process Transaction $\ll \text{extend} \gg$ Notify Fraud Department

Extend vs Include



Example


cellular network
user
place phone call
receive phone call
use scheduler
<<extend>>
<<extend>>
place conference call
receive additional call
Cellular Telephone

Use-Case Diagram Case Study [1]

Vending Machine

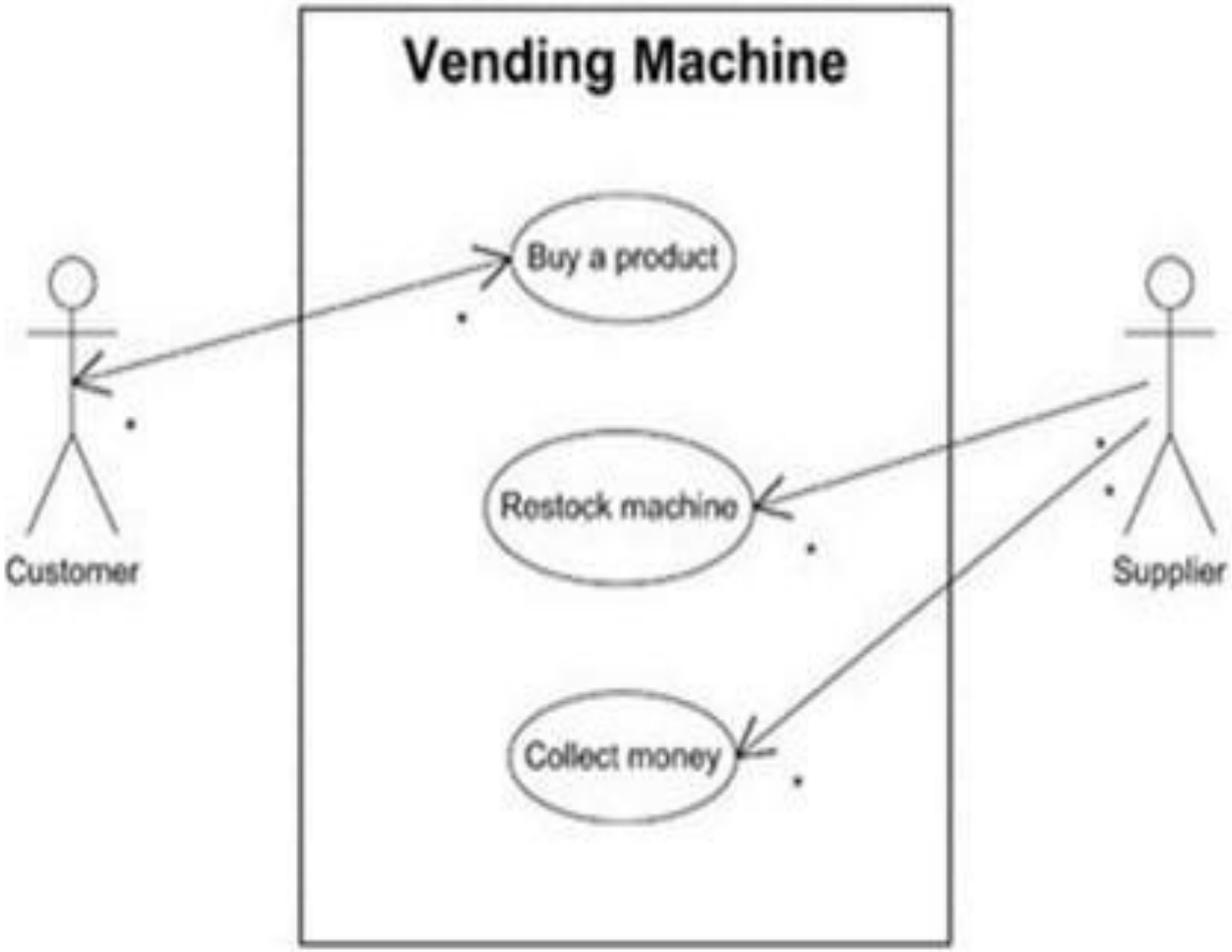
After client interview the following system scenarios were identified:

- A customer buys a product
- The supplier restocks the machine
- The supplier collects money from the machine

On the basis of these scenarios, the following three actors can be identified:

Customer; Supplier; Collector (in this case Collector=Supplier)

Use-Case Diagram Case Study [2]



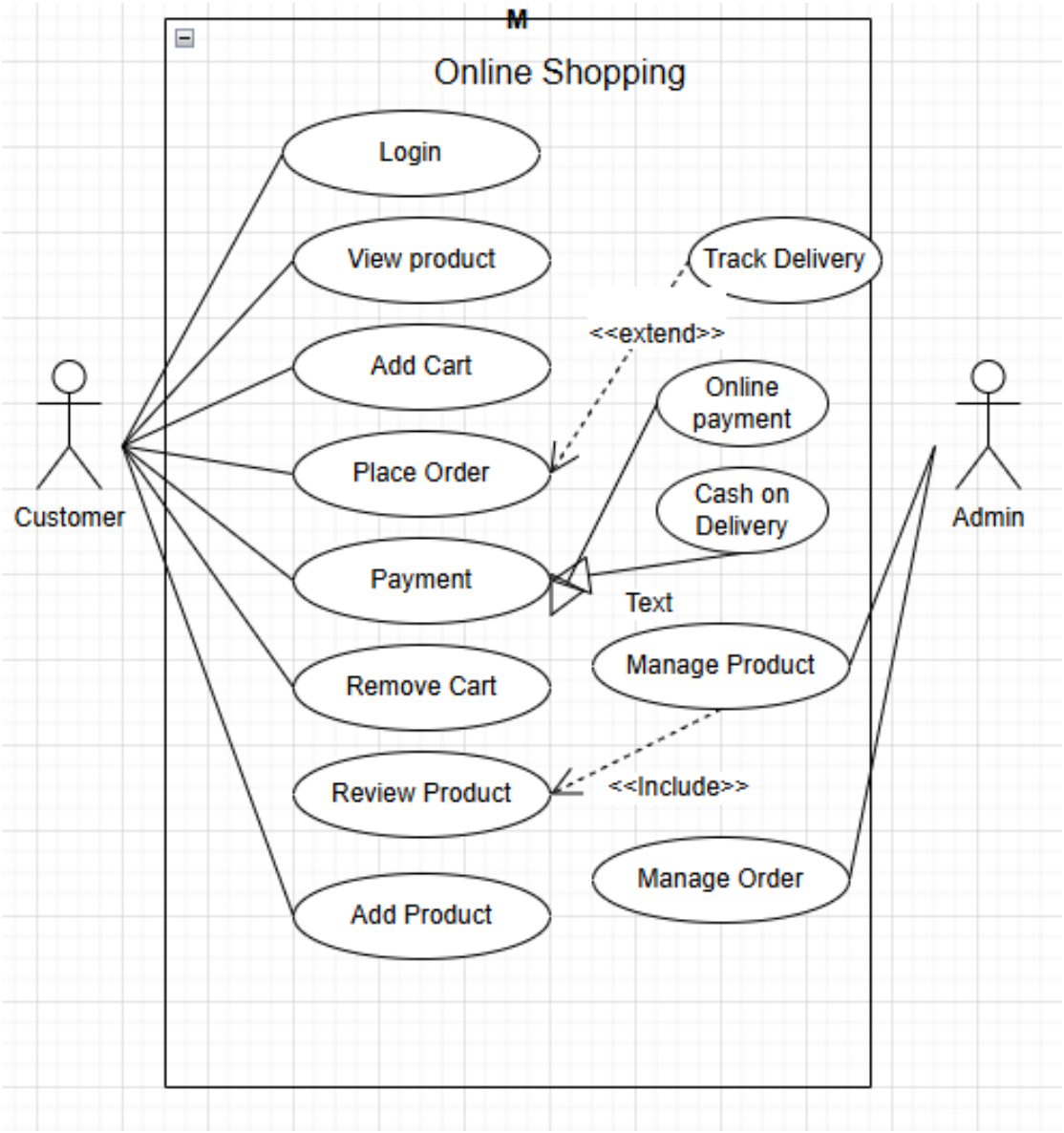
🛒 Use Case Diagram: Online Shopping System

Key Components

- **Actors:** External entities interacting with the system.
  - **Customer:** Browses, buys products.
  - **Administrator:** Manages the inventory and orders.
- **System Boundary:** The box enclosing all Use Cases, defining the system's scope (**Online Shopping System**).
- **Use Cases:** Ovals representing goals accomplished by an Actor (e.g., **Place Order**).

🛒 Use Case Diagram: Online Shopping System

Actor	Primary Use Cases (Goals)	Relationships
Customer	**View Products**, **Login**, **View Cart**, **Place Order**	**Place Order** $\ll \text{include} \gg$ **Process Payment** (Payment is a mandatory step)
Administrator	**Manage Products**, **Manage Orders**	**View Products** $\ll \text{extend} \gg$ **Review Product** (Reviewing is an optional step)


M
Online Shopping
Login
View product
Track Delivery
<<extend>>
Add Cart
Online payment
Place Order
Cash on Delivery
Payment
Text
Manage Product
Remove Cart
Review Product
<<Include>>
Manage Order
Add Product
Customer
Admin